**Project Title: Open Source Audit — Git**

**Course: Open Source Software**

**Your Name:KESHAV GOEL**

**Registration Number : 24BCE10844**

## Introduction :

Open source software plays a critical role in modern computing by enabling collaboration, transparency, and innovation. Many of the tools used in software development today are built on open-source foundations.

This report presents an audit of Git, a distributed version control system widely used in software development. The objective of this project is to understand not only the technical aspects of Git but also its philosophy, licensing model, ecosystem, and real-world impact.

The report also includes practical implementation through shell scripts, demonstrating how open-source tools interact with the Linux environment.

# PART A — Origin and Philosophy (Git)

## A1. Origin Story

Git was created in 2005 by Linus Torvalds to solve a critical problem faced by Linux developers. Initially, they relied on a proprietary version control system called BitKeeper. However, due to licensing conflicts, access to this tool was revoked.

This situation forced the development team to look for an alternative. Instead of adopting another existing tool, Linus Torvalds decided to build a new system that would be fast, reliable, and fully open source.

Git introduced a distributed model where every developer has a complete copy of the repository. This improved both performance and reliability compared to centralized systems. Over time, Git became the most widely used version control system and is now the backbone of platforms like GitHub.

## A2. License Analysis

Git is released under the GNU General Public License version 2 (GPL v2). This license ensures that users have the freedom to use, study, modify, and distribute the software.

The four key freedoms include:

- Freedom to run the software
- Freedom to study and modify the source code
- Freedom to redistribute copies
- Freedom to distribute modified versions

A key feature of GPL is “copyleft,” which requires that modified versions of the

software must also remain open source. In contrast, permissive licenses like MIT allow developers to use the code in proprietary software without sharing modifications. This makes GPL more restrictive but ensures long-term openness. Git represents “free as in freedom” rather than just free cost, meaning users have full control over how the software is used and modified.

## A3. Ethics of Open Source

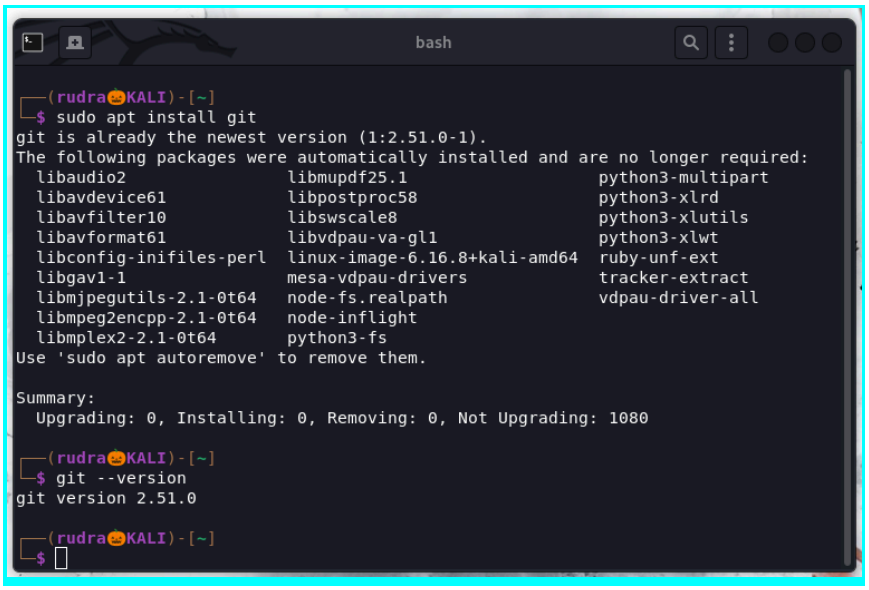
Open source promotes transparency, collaboration, and shared knowledge. It allows developers to learn from existing work and contribute improvements.

However, not all software can or should be open source. Proprietary software is important in cases where business models depend on controlled distribution.

Companies like Google use open-source technologies while generating profit. This raises ethical questions about fairness. In my view, this is acceptable as long as companies contribute back to the community.

The concept of “standing on the shoulders of giants” highlights how open source enables faster innovation by building upon existing knowledge.

# PART B — Linux Footprint of Git




Git is widely available on most Linux distributions and can be easily installed using the default package manager. On Debian-based systems such as Ubuntu, Git can be installed using commands like `sudo apt update` followed by `sudo apt install git`. Similarly, on Red Hat-based systems, it can be installed using `sudo yum install git`. After installation, the version can be verified using the `git --version` command. Installing Git through the package manager is generally preferred because it automatically manages dependencies and ensures smooth updates.

Once installed, Git follows the standard Linux filesystem structure. The main executable binary is typically located in `/usr/bin/git`, which allows users to run Git commands globally from the terminal. Configuration in Git is handled at different levels. System-wide configuration is stored in `/etc/gitconfig`, while user-specific settings are stored in the home directory as `~/.gitconfig`. Each Git repository also contains a hidden `.git` directory, which is the core of the repository and stores all commit history, branches, and metadata. Additionally, logs related to repository activities are maintained within the `.git/logs/` directory.

Unlike many server-based applications, Git does not run as a background

service. Instead, it operates as a command-line tool and runs only when a user executes a command. Git works under the permissions of the currently logged-in user, which means that access to repositories is controlled through standard Linux file permissions. This ensures that only authorized users can modify or access repository data. In collaborative environments, permissions can be managed using Linux groups or secure authentication methods such as SSH keys.

Git does not require service management tools like `systemctl` because it is not a daemon-based application. Instead, it performs tasks on demand through commands such as `git init`, `git clone`, `git add`, `git commit`, and `git status`. In some cases, Git may interact with services like SSH or web servers when working with remote repositories, but it does not function as an independent service.

Updates to Git are handled through the Linux package manager. On systems like Ubuntu, updates can be performed using commands such as `sudo apt update` and `sudo apt upgrade`. These updates include bug fixes, performance improvements, and security patches. Since Git is an open-source project, its development is maintained by a global community of contributors who continuously review and improve the code. This open development model ensures transparency, reliability, and rapid issue resolution.

During this project, Git was installed and tested on a Linux system using basic commands such as `git --version`, `git init`, and `git status`, which helped in understanding its practical behavior in a real environment.

```
┌──(rudra㉿KALI)-[~/Projects/3d_Portfolio]
└─$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/rudra/Projects/3d_Portfolio/.git/
```

# PART C — The FOSS Ecosystem (Git)

## C1. Dependencies of Git

Git, although lightweight, depends on several components of the Linux environment. It is written in C, which helps it achieve high performance and efficiency.

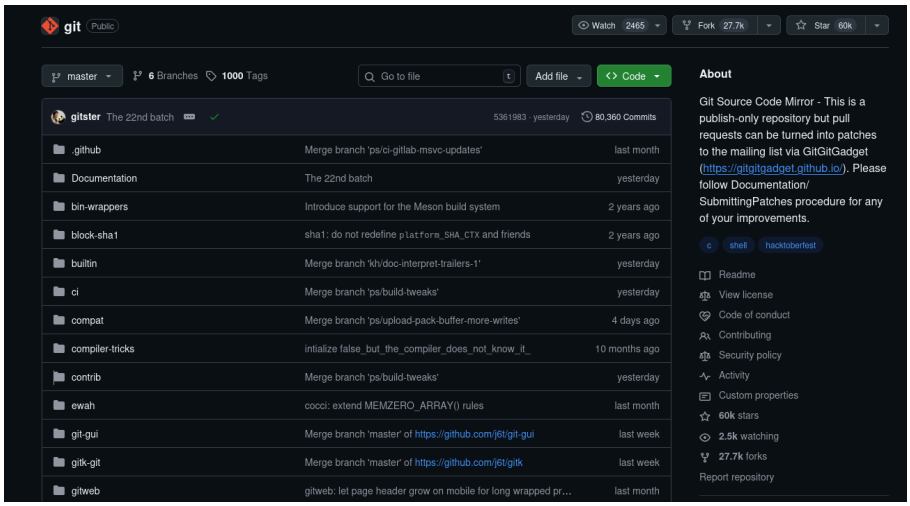
Some important dependencies include:

- Standard system libraries such as GNU C Library (glibc) ● Compression libraries like zlib for efficient data storage ● Secure communication tools like OpenSSL for HTTPS connections

Git also works with:

- SSH for secure remote access
- HTTP/HTTPS protocols for interacting with remote repositories
- Shell environments such as Bash

In addition, Git integrates with development tools like code editors and CI/CD systems, making it a central part of modern software development.




## C2. What Git Has Enabled or Inspired

Git has transformed the way developers collaborate and manage code. Its

distributed nature allows every developer to have a complete copy of the repository, which improves both speed and reliability.

Git has also enabled platforms like GitHub and GitLab, where developers can host projects, track issues, and collaborate globally.

Some major contributions of Git include:

- Distributed version control
- Branching and merging workflows
- Open collaboration through forking
- Integration with automation tools like CI/CD

Today, most modern software projects rely on Git for version control.

## C3. Git and Its Relationship with the Web / LAMP Stack

Git is not a direct part of the LAMP stack (Linux, Apache, MySQL, PHP), but it plays an important supporting role.

In web development:

- Git is used to manage source code
- Developers track changes and collaborate using repositories
- Code can be rolled back if errors occur

Git also supports deployment workflows, where code changes are automatically tested and deployed to servers. This makes it an essential tool in modern web development practices.

## C4. Community and Governance

Git has a strong global community and is not controlled by a single organization. It was originally developed by Linus Torvalds, but is now maintained by multiple contributors.

The Git community operates through:

- Mailing lists for discussions

- Online repositories for code contributions
- Documentation and forums for learning

Developers contribute by submitting patches, which are reviewed before being accepted. This ensures high-quality and reliable software.

The combination of community support and real-world usage makes Git a robust and continuously evolving project.

# PART D — Open Source vs Proprietary
# (Git vs Proprietary Alternatives)

| Git (Open Source) | Proprietary Alternative |
| --- | --- |
| **1**.Git is completely free to use, modify, and distribute. There are no licensing fees, which makes it highly accessible for individuals and organizations of all sizes. | **1**.Proprietary tools often involve licensing costs, subscription fees, or enterprise pricing models, which can increase the overall cost for organizations. |
| **2**.Since Git is open source, its code is publicly available. This allows developers worldwide to audit the code, identify vulnerabilities, and suggest fixes.<br>Transparency improves trust and security. | **2**.In proprietary software, the source code is not accessible to users. Security depends entirely on the company that owns the software, and vulnerabilities may not be visible to the public. |
| **3**.Git has strong community support, including forums, documentation, and global contributors. However, official support may not always be centralized. | **3**.Proprietary software usually offers dedicated customer support, service-level agreements (SLAs), and professional assistance, which can be beneficial for enterprises. |
| **4**.Git provides full freedom to modify the source code according to user needs. This flexibility is one of its biggest advantages. | **4**.Proprietary tools do not allow modification of the source code. Users are restricted to the features provided by the company. |
| **5.**Git is community-driven and not controlled by a single organization. Decisions are made collaboratively, which aligns with open-source values . | **5**.Proprietary software is controlled by a specific company. Decisions about features, updates, and pricing are made centrally. |

## Final Verdict

Based on this analysis, Git stands out as a powerful and flexible version control system that is suitable for both individual developers and large-scale projects.

I would strongly recommend Git for real-world deployment due to its performance, reliability, and active community support. It provides all the essential features required for modern software development while maintaining the core principles of open source.

Additionally, I would be interested in contributing to Git or similar open-source projects in the future. Contributing to such projects not only improves technical skills but also allows developers to be part of a global community that builds tools for everyone.

---

## SCRIPT 1 — System Identity Report

**CODE =>**

```
#!/bin/bash
# Script 1: System Identity Report

STUDENT_NAME="KESHAV GOEL"
SOFTWARE="Git"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
```

```
UPTIME=$(uptime -p)
DATE=$(date)
echo "==============================="
echo " Open Source Audit - $STUDENT_NAME"
echo "==============================="
echo "Software : $SOFTWARE"
echo "Kernel : $KERNEL"
echo "User : $USER_NAME"
echo "Uptime : $UPTIME"
echo "Date : $DATE"
```

## Explanation

This script displays basic system information such as kernel version, user name, uptime, and date. It uses variables and command substitution to fetch system details.

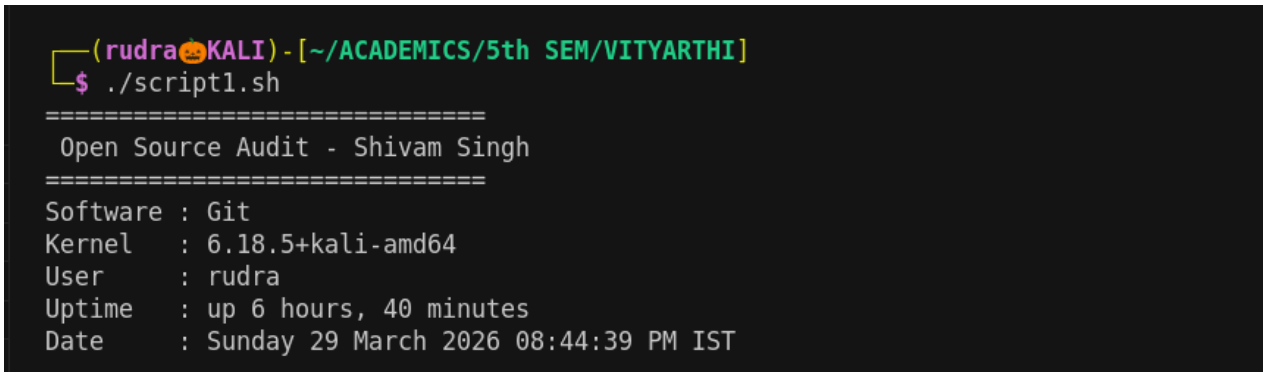



Figure: Output of Script 1

## SCRIPT 2 — FOSS Package Inspector

**CODE =>**

```
#!/bin/bash
```

```
# Script 2: Package Inspector
PACKAGE="git"

if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed."
    git --version
else
    echo "$PACKAGE is NOT installed."
fi

case $PACKAGE in
    git) echo "Git: distributed version control system" ;;
    python) echo "Python: programming language" ;;
    apache2) echo "Apache: web server" ;;
esac
```

## Explanation

This script checks whether Git is installed or not. It uses conditional statements and a case structure to display additional information about the package.

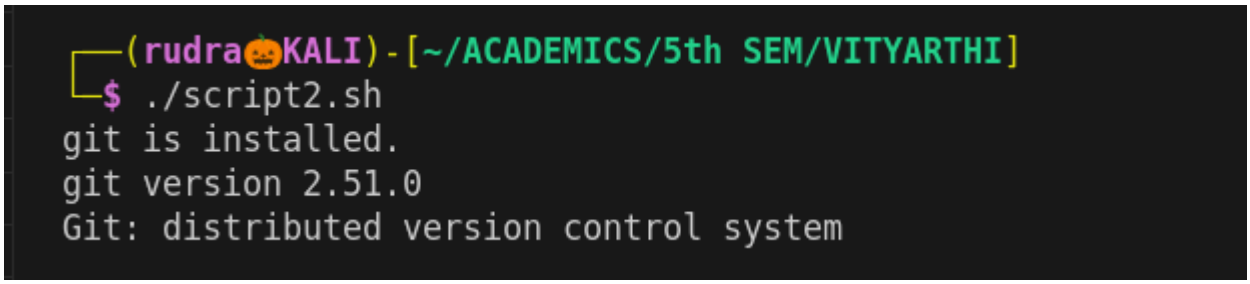



Figure: Output of Script 2

### SCRIPT 3 — Disk and Permission Auditor

**CODE =>**

```
#!/bin/bash

DIRS=("/etc" "/var/log" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
  if [ -d "$DIR" ]; then
    PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
    SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
    echo "$DIR => Permissions: $PERMS | Size: $SIZE"
  else
    echo "$DIR not found"
  fi
done
```

## Explanation

This script checks multiple directories and displays their permissions and size. It uses a for loop, condition checks, and command-line tools like `ls`, `du`, and `awk`.

```
┌──(rudra🎃KALI)-[~/ACADEMICS/5th SEM/VITYARTHI]
└─$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 24M
/var/log => Permissions: drwxr-xr-x root root | Size: 2.4G
/tmp => Permissions: drwxrwxrwt root root | Size: 2.0M
```

Figure: Output of Script 3

## SCRIPT 4 — Log File Analyzer

**CODE =>**

```
#!/bin/bash
# Script 4: Log Analyzer

LOGFILE=$1
KEYWORD=${2:-install}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "File not found"
    exit 1
fi

while read LINE; do
    echo "$LINE" | grep -iq "$KEYWORD"
    if [ $? -eq 0 ]; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times"
```

## Explanation

This script reads a log file and counts how many times a keyword appears. It uses a while loop, condition checks, and command-line arguments.

```
┌──(rudra🎃KALI)-[~/ACADEMICS/5th SEM/VITYARTHI]
└─$ ./script4.sh /var/log/alternatives.log
Keyword 'install' found 2 times
```

## SCRIPT 5 — Open Source Manifesto Generator

**CODE =>**

```
#!/bin/bash
# Script 5: Manifesto Generator

read -p "Tool you use: " TOOL
read -p "Meaning of freedom: " FREEDOM
read -p "What will you build: " BUILD

DATE=$(date)
FILE="manifesto.txt"

echo "On $DATE, I believe open source means $FREEDOM." > $FILE echo "I use $TOOL daily and aim to build $BUILD for the community." >> $FILE

echo "Manifesto saved in $FILE"
cat $FILE
```

## Explanation

This script takes user input and generates a personalized open-source manifesto. It demonstrates user input handling and file writing.

```
┌──(rudra🎃KALI)-[~/ACADEMICS/5th SEM/VITYARTHI]
└─$ ./script5.sh
Tool you use: Git
Meaning of freedom: Creativity
What will you build: an open source tool for devs
Manifesto saved in manifesto.txt
On Sunday 29 March 2026 08:49:45 PM IST, I believe open source means Creativity.
I use Git daily and aim to build an open source tool for devs for the community.
```

# CONCLUSION :

This project provided a comprehensive understanding of Git as an open-source software and its importance in modern software development. It highlighted how Git was developed to solve real-world problems and how it follows the principles of open-source philosophy such as transparency, collaboration, and freedom.

Through the analysis of its license, ecosystem, and comparison with proprietary alternatives, it became clear that open-source software offers flexibility, security, and community-driven innovation. Git, in particular, demonstrates how a well-designed tool can be widely adopted across both individual and enterprise-level projects.

The practical implementation using shell scripts helped in understanding how Git interacts with the Linux environment and how basic system tasks can be automated. This hands-on experience strengthened both theoretical and practical knowledge.

Overall, Git proves to be a powerful, reliable, and essential tool in modern computing, and this project has enhanced my understanding of open-source systems and their real-world impact.